

Tetherless: Decentralized Encrypted Chat Application
A

Report submitted in partial fulfilment of the requirements for the
degree of
B.Tech.

In
Computer Science & Engineering

By
Aryan Singh (2401640100321)
Ayushi Srivastava (2401640100406)
Arushi Verma (2401640100310)
Ayushi Singh (2401640100404)
Arushi Dwivedi (2401640100308)
Avinay Yadav (2401640100356)

Under the guidance of
Rajendra Singh
Assistant Professor
Project Id: 26_CS_3N_03



Pranveer Singh Institute of Technology, Kanpur
Dr A P J A K Technical University
Lucknow

November-2026

DECLARATION

This is to declare that the report entitled “Tetherless: Decentralized Encrypted Chat Application”, submitted by Aryan Singh (2401640100321), Ayushi Srivastava (2401640100406), Arushi Verma (2401640100310), Ayushi Singh (2401640100404), Arushi Dwivedi (2401640100308), Avinay Yadav (2401640100356) in partial fulfilment of the requirement for the award of the degree B.Tech. in Computer Science and Engineering at Pranveer Singh Institute of Technology, Kanpur, affiliated to Dr. A P J A K Technical University, Lucknow, comprises only our own work, and due acknowledgement has been made in the text to all other material used.

Date:

Aryan Singh (2401640100321)
Ayushi Srivastava (2401640100406)
Arushi Verma (2401640100310)
Ayushi Singh (2401640100404)
Arushi Dwivedi (2401640100308)
Avinay Yadav (2401640100356)

CERTIFICATE

This is to certify that the report entitled “Tetherless: Decentralized Encrypted Chat Application”, which is submitted by **Aryan Singh (2401640100321), Ayushi Srivastava (2401640100406), Arushi Verma (2401640100310), Ayushi Singh (2401640100404), Arushi Dwivedi (2401640100308), Avinay Yadav (2401640100356)** in partial fulfilment of the requirements for the award of the degree B.Tech. in Computer Science & Engineering at Pranveer Singh Institute of Technology, Kanpur, affiliated to Dr. A P J A K Technical University, Lucknow, is a record of the candidates' own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Signature:

Rajendra Singh
Assistant Professor
CSE Department,
PSIT, Kanpur

Signature:

Dr. Nand Kishore Sharma
Associate Professor & Head
CSE Department,
PSIT, Kanpur

ACKNOWLEDGEMENT

*It gives us great sense of pleasure to present our project undertaken during B.Tech. Third Year (Session: 2026-27). We owe a special debt of gratitude to our project supervisor, **Rajendra Singh, Assistant Professor, Department of Computer Science and Engineering, Pranveer Singh Institute of Technology, Kanpur**, for their constant support and guidance throughout the course of our work. Their sincerity, thoroughness, and perseverance have been a constant source of inspiration for us.*

*We also take the opportunity to acknowledge the contribution of **Dr. Nand Kishore Sharma, Head, Department of Computer Science & Engineering, Pranveer Singh Institute of Technology, Kanpur**, for his full support and assistance during the development of the project.*

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not least, we acknowledge our friends for their contribution to the completion of the project.

Signature

Name: Aryan Singh

Roll No.: 2401640100321

Signature

*Name: Ayushi
Srivastava*

*Roll No.:
2401640100406*

Signature

Name: Arushi Verma

Roll No.: 2401640100310

Signature

*Name: Ayushi
Singh*

*Roll No.:
2401640100404*

Signature

Name: Arushi Dwivedi

Roll No.: 2401640100308

Signature

*Name: Avinay
Yadav*

*Roll No.:
2401640100356*

ABSTRACT

Mainstream messengers increasingly encrypt message content, yet they still learn who talks to whom, tie identity to a telephone number, and ask users to accept privacy claims that cannot be inspected. This project designs and implements Tetherless, a self-hostable end-to-end encrypted chat system in which the relay server is untrusted by construction and a user's identity is a key that the user generates and holds.

The system consists of a relay, a shared core library and desktop clients, the latest being an Electron interface over a headless Java engine. Peers agree a session key through a signed Diffie-Hellman exchange carried over pinned TLS 1.3, protect each message with AES-256-GCM, reject replayed frames with a sliding window, and verify one another through safety numbers. History kept on the device is encrypted under a key stretched from the user's passphrase with PBKDF2. The design covers the threat model, the wire protocol, session and storage handling, and the confinement of the Electron renderer.

The system passes 381 JVM tests and 18 renderer tests, a 17-check end-to-end run and a 24-check run in the real application window. Four tests that check POSIX file permissions were skipped on Windows. On one machine over loopback, a message reaches its peer in a median of about 15 ms. The evaluation found and corrected nine principal defects, among them nonce reuse across directions and a relay private key shipped inside its own package.

The system provides no forward secrecy beyond the handshake and no metadata privacy, has not been independently audited, and has not been tested with users. These limits are stated plainly, together with proposed future work.

Table of Contents

S NO	CHAPTER	Page No
1	Declaration	ii
2	Certificate	iii
3	Acknowledgement	iv
4	Abstract	v
5	Table of Contents	vi
6	List of Figures	viii
7	List of Tables	ix
8	Introduction	1
	1.1 Background and Motivation	1
	1.2 Problem Statement	2
	1.3 Objectives	3
	1.4 Scope of the Project	3
	1.5 Literature Survey	4
	1.6 Technologies Used	6
	1.7 Organisation of the Report	6
9	Proposed Methodology	8
	2.1 Development Approach	8
	2.2 Requirements	9
	2.3 System Architecture	10
	2.4 Threat Model	11
	2.5 Cryptographic Design	12
	2.6 Wire Protocol	14
	2.7 Connection Establishment and Handshake	16
	2.8 Message Processing and Session Management	18
	2.9 Relay Server Design	20
	2.10 Client Design: Storage, Keys and Delivery State	21
	2.11 The Electron Client	23
	2.12 User Interface Design	27
	2.13 Implementation Summary	29
10	Results and Discussions	31
	3.1 Test Strategy and Outcomes	31
	3.2 Security Evaluation	32
	3.3 Performance Measurements	33
	3.4 The Running Application	34
	3.5 Discussion	37

S NO	CHAPTER	Page No
	3.6 Limits of the Results	38
11	Conclusion and Future Scope	39
	4.1 Conclusion	39
	4.2 Limitations	39
	4.3 Future Scope	40
12	References	41

LIST OF FIGURES

Fig. 1.1	Trust model: what each party can and cannot see	2
Fig. 2.1	Overall architecture: clients, relay and the shared core library	10
Fig. 2.2	Module structure of the repository and the dependencies between modules	11
Fig. 2.3	Layout of a frame and of the message it carries	15
Fig. 2.4	Registration, mutual introduction, signed key exchange and first message	17
Fig. 2.5	Pipeline for an outgoing message (top) and an incoming message (bottom)	18
Fig. 2.6	Structure of the GCM nonce and the replay window	19
Fig. 2.7	Session states and transitions	20
Fig. 2.8	Key derivation and the contents of the profile directory	22
Fig. 2.9	Process architecture of the Electron client	24
Fig. 2.10	Commands, replies and events between the renderer, the main process and the engine	25
Fig. 3.1	Number of automated tests per suite	32
Fig. 3.2	Median and 95th percentile latency of the measured operations	34
Fig. 3.3	A conversation with message states and day grouping (dark theme)	35
Fig. 3.4	Replies showing the quoted original	35
Fig. 3.5	Safety-number panel and the verified shield	36
Fig. 3.6	Search results with highlighted matches	36
Fig. 3.7	Settings panel (dark theme)	37
Fig. 3.8	The same conversation in the light theme	37

LIST OF TABLES

Table 1.1	Comparison of Tetherless with representative messengers.....	5
Table 1.2	Technologies used in the project.....	6
Table 2.1	Summary of functional requirements	9
Table 2.2	Summary of non-functional requirements.....	9
Table 2.3	Threats considered, the control for each, and what remains	12
Table 2.4	Cryptographic primitives and parameters	12
Table 2.5	Fields of an encoded message	15
Table 2.6	Message types	16
Table 2.7	Commands accepted by the engine	25
Table 2.8	Measures that confine the renderer	27
Table 2.9	Principal colours of the two themes	28
Table 2.10	Size of each module (non-blank lines, excluding tests)	30
Table 3.1	Automated test results	31
Table 3.2	Principal defects found and corrected during development	32
Table 3.3	Measured latencies on the test machine (milliseconds)	34

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Text messaging has become the default channel for personal and professional conversation. Almost all of it is carried by a small number of platforms that sit in the middle of every exchange. Over the last decade the content of those conversations has increasingly been protected by end-to-end encryption, in which a message is encrypted on the sender's device and can be decrypted only on the recipient's. This is a genuine improvement: it means that the operator of the service, and anyone who compromises the operator, cannot simply read what people write.

Encrypting the content, however, does not settle the question of what else the platform learns. Three problems remain, and they motivate this project.

- **Metadata is rarely protected.** Even when message bodies are unreadable, the operator typically keeps a record of who talked to whom, when, how often, and how much. This social graph is frequently more revealing than any single message, and it is usually retained.
- **Identity is rented from the platform.** Most mainstream messengers bind an account to a telephone number. A number is a poor secret: it can be recycled by a carrier, taken over through a SIM swap, or simply withdrawn by an operator. It also excludes people who cannot or do not wish to attach their number to a messaging account.
- **The guarantees are asserted, not demonstrated.** Users are asked to believe a privacy claim that they have no practical means to inspect. Systems whose source, protocol and test evidence are not available for scrutiny can only be trusted, not verified.

Tetherless is a chat application built to examine a narrow and specific alternative. Identity is a cryptographic key that the user generates and holds. The server that routes messages is treated as hostile and is designed to be unable to read them. The whole system, including that server, can be operated by the people who use it. Most importantly for an academic project, every security property the system claims is tied to an automated test, and every property it does not provide is stated plainly in the project's documentation.

1.2 Problem Statement

The problem addressed by this project is stated as follows:

To design and implement a complete, self-hostable, end-to-end encrypted chat system, consisting of a relay server and a desktop client, in which the relay is untrusted by construction, a user's identity is a key they hold rather than an account they are issued, two users can verify that they are not being intercepted, and every security claim is backed by evidence and every limitation is disclosed.

Three things make this harder than it first appears. The first is that cryptographic components that are individually sound can be composed into a system that is not: nonce reuse, a replay window with a gap, or a handshake that does not authenticate the party it is talking to are each ordinary-looking mistakes with severe consequences. The second is that a messaging client is also an ordinary graphical application, and a security product that is unpleasant to use is not used. The third is that the same protocol must be implemented identically on more than one platform, since a disagreement about bytes shows up as an intermittent failure rather than as an error at build time.

Fig. 1.1 summarises the trust model that the proposed system adopts in response. The two clients hold the keys and see plaintext. The relay between them is assumed hostile: it sees only ciphertext and the routing information needed to deliver it, and it can neither read nor forge a message.

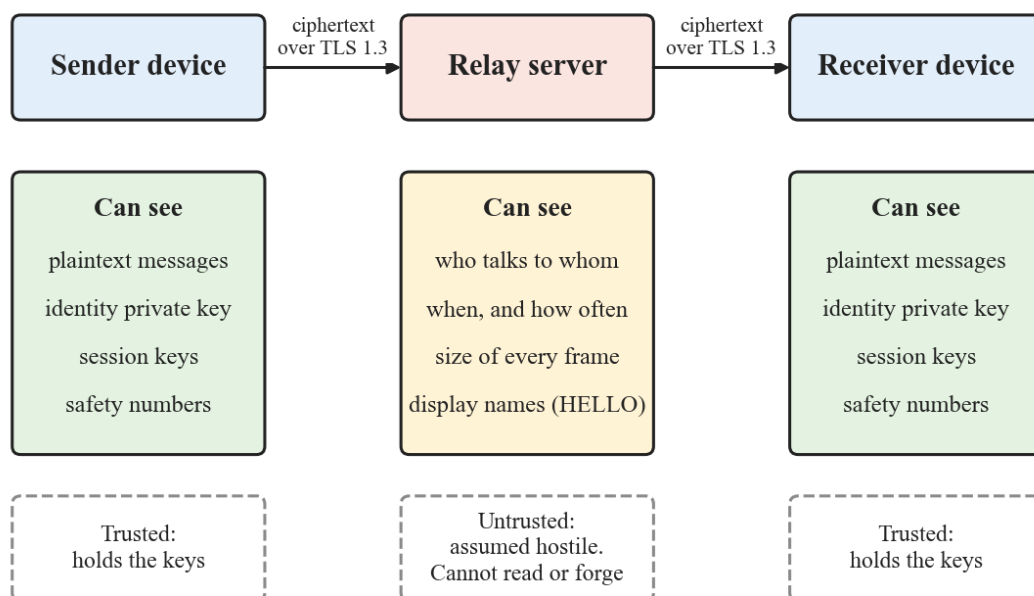


Fig. 1.1: Trust model: what each party can and cannot see

1.3 Objectives

The objectives of the project are the following.

1. To design a wire protocol with an explicit, bounded binary encoding, in which every inter-client frame is signed and every variable-length field has a size limit checked before allocation.
2. To implement an authenticated key agreement, so that a relay that substitutes keys during the exchange is detected rather than trusted.
3. To implement message encryption with counter-based nonces, replay protection and bounded key lifetime, and to demonstrate by testing that a nonce is never reused under one key.
4. To implement a relay server that routes ciphertext by recipient identifier, verifies who is registering, limits abuse, and holds no key material and no message content.
5. To implement a desktop client that stores history encrypted at rest under a key derived from a passphrase that is never itself stored.
6. To let two users verify each other's identity through a human-comparable safety number, and to block sending when a known contact's key changes.
7. To build a modern, responsive user interface for the desktop client, without moving any cryptography or key handling into the part of the program that renders untrusted text.
8. To document the threat model, the guarantees and, equally, the limitations of the system, and to validate the claims with automated tests and measurements.

1.4 Scope of the Project

The scope is deliberately narrow. A system that tries to match a mainstream messenger feature for feature would not be finished, and would be difficult to analyse. The project is therefore bounded as follows.

Within scope: one-to-one text messaging between users who can already exchange an identifier; a self-hostable relay; identity creation and passphrase-protected key storage; encrypted local history; delivery and read state; safety-number verification with key-change detection; search over local history; replies that quote an earlier message; desktop notifications; light and dark appearance; and an automated test suite with a reproducible end-to-end harness.

Outside scope: group messaging, voice and video, file and image transfer, a directory or

discovery service, message backup to a server, use of one identity on several devices, and any form of account recovery by the operator. The last is not an omission but a consequence of the design. A service that can restore a user's identity is a service that can impersonate that user, which is exactly what the system is built to prevent.

The security properties that this system does not provide are listed in Section 4.2 and in the project's own security document. They include per-message forward secrecy, protection of metadata from the relay, and deniability.

1.5 Literature Survey

1.5.1 Cryptographic foundations

The ideas on which the system rests are well established. Diffie and Hellman showed in 1976 that two parties can agree on a shared secret over a public channel [1]. That result is the basis of the key agreement used here. Because an unauthenticated exchange is open to an active attacker who sits between the parties, the exchange must be bound to long-term identities; this project does so with RSA signatures [2]. The specific group used is the 2048-bit modular exponential group standardised in RFC 3526 [3].

The agreed secret is not used directly. It is passed through the HMAC-based key derivation function specified in RFC 5869 [4] to produce an AES-256 key. Messages are protected with AES in Galois/Counter Mode, which provides confidentiality and integrity together; the standard governing it is NIST SP 800-38D [5]. That standard is also the source of the most important operational constraint in this project: GCM is catastrophically weak if a key and nonce pair is ever repeated, which is why nonce construction receives particular attention in Chapter 2. Keys that protect data at rest are derived from a passphrase with PBKDF2, specified in RFC 8018 [6] and discussed in NIST SP 800-132 [7], so that every guess an attacker makes against a stolen database is made expensive. Transport security is provided by TLS 1.3 [8].

1.5.2 Secure messaging protocols

Off-the-Record messaging, introduced by Borisov, Goldberg and Brewer in 2004 [9], was among the first protocols to treat a private conversation as something that should leave no cryptographic proof of who said what. It introduced the idea of deniable authentication and of keys that are discarded as a conversation proceeds. Those ideas were carried into the Signal protocol, whose two central components are the Extended Triple Diffie-Hellman key agreement, which allows a session to be established with a user who is offline [10], and the

Double Ratchet, which derives a fresh key for every message and so gives both forward secrecy and recovery after a compromise [11]. Cohn-Gordon and colleagues published a formal security analysis of the Signal protocol in 2017 [12], and Unger and colleagues surveyed the wider design space of secure messaging in 2015 [13].

More recent work addresses groups. The Messaging Layer Security protocol, standardised as RFC 9420 [14], maintains a shared group key with efficient membership changes. These designs are considerably more capable than the one described in this report, and the report is explicit about where the gap lies.

1.5.3 Deployed systems and what they trade

Deployed messengers make different trade-offs between security, convenience and operator control. Table 1.1 compares Tetherless with a representative set. The judgements are drawn from each project's public documentation, which changes over time, and they are given to locate this project rather than to rank products.

Table 1.1: Comparison of Tetherless with representative messengers

Property	Tetherless	Signal	WhatsApp	Telegram	Matrix	Briar
End-to-end encrypted by default	Yes	Yes	Yes	No (opt-in Secret Chats)	Yes in private rooms	Yes
Needs a phone number	No	Yes	Yes	Yes	No	No
Can be self-hosted	Yes	No	No	No	Yes	No central server
Per-message forward secrecy	No	Yes	Yes	Partial	Yes	Yes
Group messaging	No	Yes	Yes	Yes	Yes	Yes
Multiple devices	No	Yes	Yes	Yes	Yes	No
Operator sees who talks to whom	Yes	Reduced	Yes	Yes	Yes (homeserver)	No operator

The comparison shows where Tetherless stands. It is ahead on two properties that matter for the problem stated above: it needs no phone number, and the entire system, including the relay, can be run by its users. It is behind on forward secrecy, groups and multiple devices, and these are stated as limitations rather than hidden. The operator of its relay can see who talks to whom, which is also true of most of the others in the table; the relay can nevertheless neither read nor forge a message, which is the property the design concentrates on.

1.5.4 The gap this project addresses

Production messengers are large, rapidly changing systems, and their security claims are best examined from the outside. Research protocols are usually presented as specifications and proofs without a complete application around them. Between the two there is room for a system that is small enough to be read and understood end to end, complete enough to be used, and documented so that its claims can be checked. Tetherless is intended to occupy that position. It does not try to out-feature the mature systems; its contribution is a faithful, tested and honestly described implementation of a sound core design, together with an account of the defects found along the way.

1.6 Technologies Used

The technologies are listed in Table 1.2 with the role each plays. Two of the choices are worth explaining. The protocol and cryptographic core is written in Java at language level 8, because the same code must run on the desktop Java virtual machine and on Android, and Java 8 is the common denominator. The graphical client is written in TypeScript with React and runs inside Electron, which gives access to a mature ecosystem for building interfaces, while the cryptography stays in the Java process (Section 2.11).

Table 1.2: Technologies used in the project

Layer	Technology	Role
Protocol and crypto core	Java 8, Java Cryptography Architecture	Codec, AES-GCM, Diffie-Hellman, RSA, sessions, TLS pinning
Relay server	Java 8, TLS sockets, SLF4J and Logback	Registration, routing, rate limiting, metrics
Client logic	Java 8, SQLite (JDBC), PKCS#12 keystores	Local store, key management, outbox, delivery state
Headless engine	Java 8, Gson	JSON command and event interface over standard streams
Original desktop UI	Java Swing, FlatLaf	First graphical client, retained as a reference
Desktop UI	Electron 31, React 18, TypeScript 5, Vite 5	Sandboxed renderer, preload bridge, process supervision
Build and quality	Gradle 8, npm, Checkstyle, SpotBugs with find-sec-bugs	Build, static analysis gates, packaging
Testing	JUnit 4, Node.js test runner, Chrome DevTools Protocol	Unit, integration, end-to-end and user-interface checks

1.7 Organisation of the Report

The remainder of the report is organised as follows. Chapter 2 describes the proposed methodology: the development approach, the requirements, the architecture and threat model,

the cryptographic and protocol design, the relay and client designs, the persistent storage, and the design of the Electron client and its user interface. Chapter 3 presents the results: the test strategy and its outcomes, the security evaluation, measurements of the running system, screenshots of the application, and a discussion that relates the results back to the objectives and is candid about their limits. Chapter 4 concludes the report, restates the limitations and sets out the future scope. The references follow.

CHAPTER 2

PROPOSED METHODOLOGY

2.1 Development Approach

The project was developed iteratively, with the security-critical core built first and the user interface built last. Several practices were adopted deliberately, because in a cryptographic application the usual signs that software is working, such as a screen that renders and a message that arrives, say very little about whether it is safe.

- **Tests are written first and watched failing.** A test that has never been seen to fail may be asserting nothing. The rule adopted was that a new behaviour starts with a test that is run and observed to fail for the expected reason, and only then is the implementation written. Characterization tests, which pin behaviour that already exists, are the one labelled exception.
- **Guards are validated by mutation.** For the protections that matter most, such as the binding between an identifier and its key, signature verification, and the replay-window floor, the guard was temporarily removed and the corresponding test was confirmed to fail. A guard whose removal changes nothing is not being tested.
- **Static analysis is a gate, not a report.** Checkstyle and SpotBugs with the find-secbugs plugin run on every build of every Java module, and a finding at high confidence fails the build. The project keeps a single, narrowly scoped suppression with a stated reason.
- **One protocol implementation.** The codec, the cryptography and the session logic live in one module that every client and the relay share. A set of seventeen frozen wire-format vectors is checked by the tests so that a change to the encoded bytes cannot happen unnoticed.
- **Decisions and limitations are written down.** Twelve architecture decision records capture why the load-bearing choices were made and, where it applies, why they now look wrong. A separate security document lists the threat model, the guarantees and the limitations.
- **Planned, reviewed increments.** The later user-interface work was carried out against written implementation plans, and each increment was reviewed against its requirements before the next began.

The same discipline was applied to the final stage. Behaviour of the finished user interface

was checked by driving the real application window and asserting on what it displayed, rather than only by confirming that the code compiled (Section 3.4).

2.2 Requirements

The requirements were derived from the problem statement and are recorded, numbered and testable, in the project's software requirements document. Table 2.1 summarises the functional requirements and Table 2.2 the non-functional ones.

Table 2.1: Summary of functional requirements

ID	Requirement	Primary evidence
FR-1, FR-3	Generate an RSA identity key pair locally; derive the peer id from the public key	Engine and PeerId tests
FR-5	Refuse a wrong passphrase without saying which check failed	Engine test
FR-9, FR-10	Sign key-exchange frames; discard any frame whose signature does not verify	Session and relay tests
FR-12	Report a changed identity key for a known peer and block sending	Adversarial relay tests
FR-13, FR-14	Encrypt bodies with AES-256-GCM; never reuse a nonce under one key	Nonce-reuse test
FR-15, FR-16	Reject counters at or below the replay floor and repeated counters	Replay-window tests
FR-19	Refuse to send without a session; no plaintext fallback	Secure-chat tests
FR-22	A delivery state never moves backwards (read does not become delivered)	Status and repository tests
FR-23, FR-24	Encrypt stored bodies; derive the key with PBKDF2 at 210,000 iterations	Repository and key tests
FR-27, FR-29	Use TLS 1.3 with a pinned certificate; report connected only after the relay acknowledges	Connection tests
FR-32, FR-33	Engine speaks only over its standard streams; stdout carries frames and nothing else	Engine run and user-interface check
FR-38 to FR-40	Relay routes by receiver id, verifies registration, rejects forged sender ids	Relay and integration tests
UI	Search, replies with quotes, notifications, settings, theme, safety-number verification	Real-window checks (Section 3.4)

Table 2.2: Summary of non-functional requirements

Area	Requirement
Security	No key, nonce or message body is logged; all randomness for keys, nonces and salts is cryptographically secure
Security	No Java serialization on the wire; every failure of a security check fails closed
Security	The user-interface renderer is sandboxed, has no Node.js access, holds no key material and never renders message text as HTML

Area	Requirement
Performance	Cryptographic and input/output work never runs on the user-interface thread; unlock completes in well under two seconds
Reliability	Loss of the relay connection loses no unsent message; the client reconnects with backoff without user action
Reliability	A malformed frame from a peer never drops the connection
Maintainability	Java modules compile at language level 8; Checkstyle and SpotBugs pass at high confidence
Maintainability	Exactly one protocol implementation, shared by every client, checked by frozen vectors

2.3 System Architecture

The system has three kinds of component, shown in Fig. 2.1: two or more clients, a relay server, and a shared core library. A client holds the user's keys, performs all encryption and decryption, and stores history. The relay accepts TLS connections, learns which client identifier is on which connection, and copies frames from one connection to another. It keeps no state about conversations and stores nothing. The shared library, named core-shared, contains the wire codec, the cryptographic primitives, signing and verification, the session logic with its replay window, and the transport with its certificate pinning. Both the clients and the relay use it, so there is exactly one implementation of the protocol.

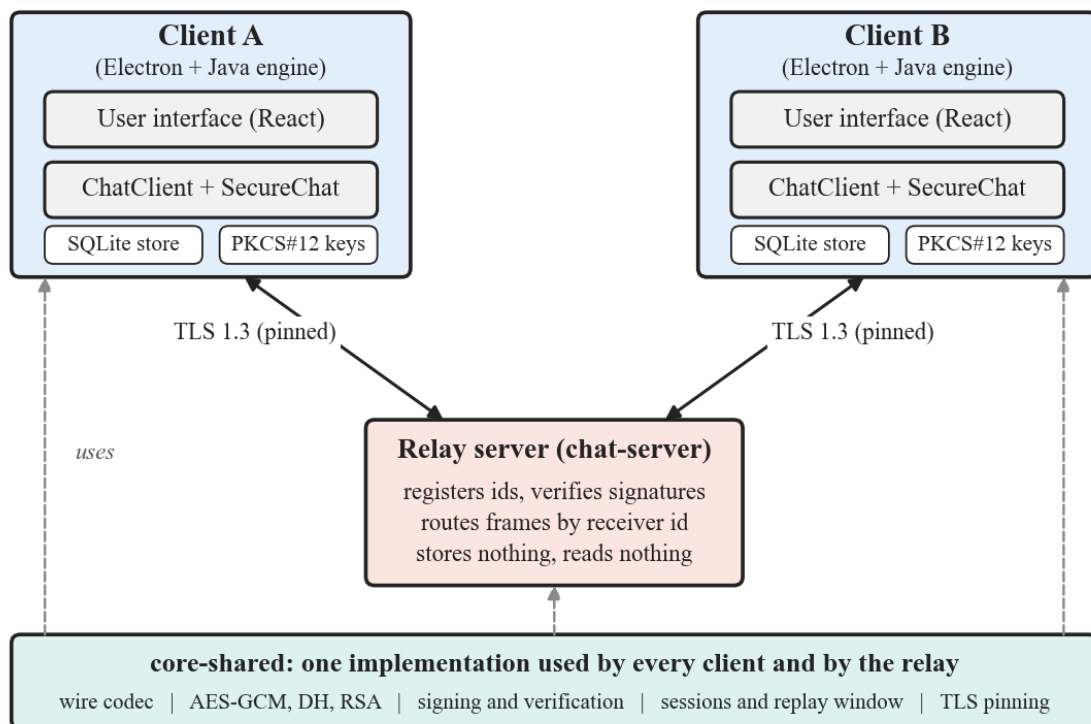


Fig. 2.1: Overall architecture: clients, relay and the shared core library

The decision to use a relay and not a direct peer-to-peer connection was taken early and

recorded. Direct connection between two consumer machines requires traversing network address translation, which in practice needs STUN and TURN servers and a relaying fallback for the cases in which hole punching fails. That fallback is the relay design, so a peer-to-peer path would have been additional machinery on top of what had to exist anyway. The cost is a single point of failure and a single point of observation, and this is stated openly in the limitations.

The repository is organised as a set of modules, shown in Fig. 2.2. The relay, the original Swing client, the headless engine and the early Android client all depend on the shared core. The Electron application is built by npm and not by Gradle; it communicates with the engine over a pipe and has no compile-time dependency on any Java code. A Compose Multiplatform scaffold was started and then abandoned in favour of Electron, and is kept in the repository only as a record.

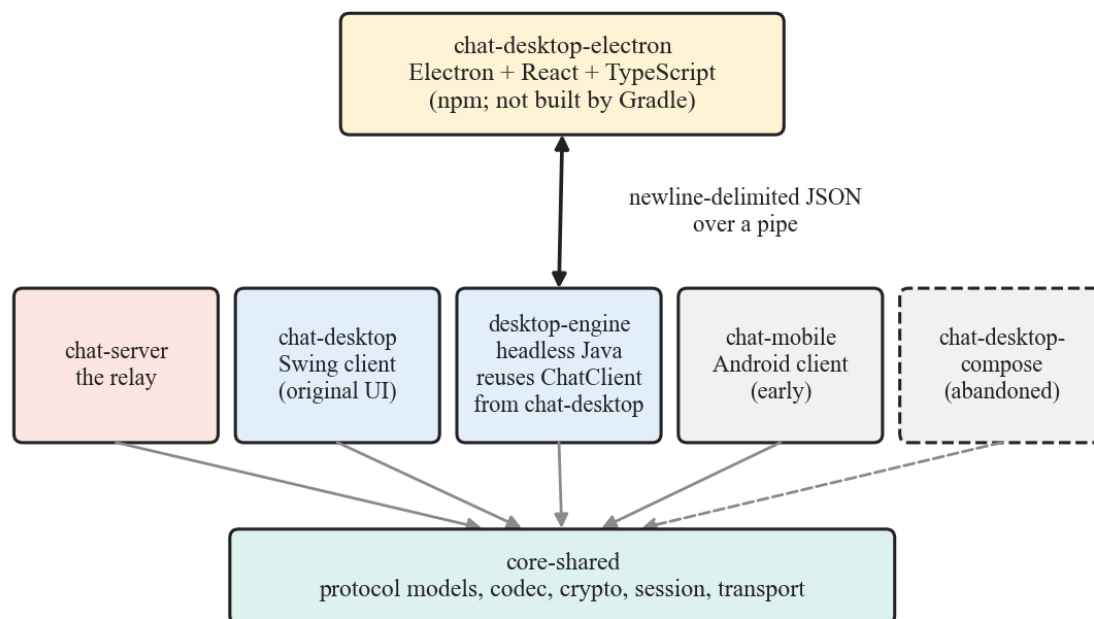


Fig. 2.2: *Module structure of the repository and the dependencies between modules*

2.4 Threat Model

The design assumes that the relay is hostile. It is expected to read everything it can, alter anything it can and lie about anything it can, and the system is required to leak nothing beyond routing metadata. The transport between client and relay is treated as untrusted as well, with TLS applied in addition to end-to-end encryption, not in place of it. The clients are trusted: they hold keys and see plaintext, and a compromised client is outside the scope of

what any messenger can defend against. Fig. 1.1 shows who can see what. Table 2.3 lists the threats considered and the control that answers each.

Table 2.3: Threats considered, the control for each, and what remains

Threat	Control	Residual risk
T1 Passive relay operator reads traffic	Bodies are AES-256-GCM ciphertext under a key the relay never holds; reply metadata travels inside the ciphertext	Routing metadata remains visible
T2 Active relay substitutes keys (man in the middle)	Key exchange is signed with long-term identity keys; the id-to-key binding is checked on every HELLO; a changed key is reported and blocks sending	Relies on the user comparing safety numbers on first contact
T3 Network attacker	TLS 1.3 with a pinned certificate protects metadata and link integrity	Content is not protected by TLS, only by end-to-end encryption
T4 Replay and reordering	Counter inside the nonce; window floor at highest minus 1024; timestamps within five minutes	Frames more than 1024 behind are dropped even if genuine
T5 Malicious payload from a peer	Explicit length-prefixed codec; per-field caps checked before allocation; frames over 1 MiB rejected	None known
T6 Device compromise	Bodies encrypted at rest under a passphrase-derived key; keys generated in process; owner-only file permissions	A live compromised process is not defended against

2.5 Cryptographic Design

Table 2.4 lists the primitive chosen for each purpose. All of them come from the standard Java Cryptography Architecture; the project implements no cryptographic algorithm of its own, and writes only the code that combines them.

Table 2.4: Cryptographic primitives and parameters

Purpose	Choice	Parameters
Message encryption	AES-GCM	256-bit key, 96-bit nonce, 128-bit tag
Key agreement	Finite-field Diffie-Hellman	RFC 3526 Group 14, 2048-bit
Session key derivation	HKDF-SHA256	32-byte output, fixed salt and context string
Identity and signatures	RSA with SHA-256	2048-bit key, SHA256withRSA
Peer address	SHA-256 of the public key	First 128 bits, as 32 hexadecimal characters
At-rest key derivation	PBKDF2-HMAC-SHA256	210,000 iterations, random per-profile salt
Transport	TLS	Version 1.3 only, certificate pinned

2.5.1 Key agreement

Each pair of peers derives a shared secret using Diffie-Hellman over the 2048-bit group of RFC 3526. The group was chosen over the more modern X25519 for a recorded reason: both

clients had to implement the same primitive identically, one of them is an Android client with a minimum platform level at which X25519 was not available, and two implementations of one primitive are the kind of duplication the project exists to avoid. The cost is a larger and slower exchange, which is not noticeable on a path used once per conversation. The decision carries an explicit trigger for revisiting it, and migration to a curve is identified as future work in Chapter 4.

Two details prevent subtle failures. First, a received public value is validated before use: it must lie strictly between 1 and p minus 1, and raised to the order of the subgroup it must equal 1, which rejects small-subgroup points. Second, the shared secret is left-padded to 256 bytes before it is passed to the key derivation function. The secret is an integer, so it is shorter than 256 bytes about once in 256 exchanges, and the Java and Android providers disagreed about whether to keep the leading zero. Without the padding, two platforms derived different keys and the handshake failed intermittently from code that looked correct on both sides. The padding is pinned by committed test vectors.

2.5.2 Key derivation and message encryption

The padded secret is passed through HKDF with SHA-256 to produce a 32-byte AES key. The derivation uses a fixed all-zero salt and a context string that names the protocol version, so a key derived for this protocol cannot be confused with one derived for another purpose. The HKDF implementation is checked against the published test vectors of RFC 5869 and not against its own output, so a defect in the implementation cannot be recorded as the expected answer.

Messages are encrypted with AES-256-GCM. The nonce is not random. It is built from a four-byte direction value followed by an eight-byte counter, shown in Fig. 2.6. A random 96-bit nonce collides often enough over a long session to matter, and GCM does not survive a repeated key and nonce pair: reuse reveals the exclusive-or of the two plaintexts and allows forgery. A counter that increases by one for every message guarantees uniqueness under one key, provided the two peers never use the same counter space. They share one derived key, so the direction value differs between them: the peer whose identifier sorts lower transmits on 1 and the other on 0. Early in development both peers used the same direction value, so each side's n -th message reused a key and nonce pair. This was found while checking a claim for the security document, not by a user, and is now pinned by a dedicated test. The receiver also checks the direction value it is given and drops a frame that does not carry the value the sending peer should be using.

2.5.3 Identity and signatures

A peer's address is derived from its identity key: the first sixteen bytes of the SHA-256 digest of the X.509-encoded public key, written as 32 hexadecimal characters. There are therefore no usernames, no registration and no directory, and there is nothing to squat or to compromise. An identifier cannot be chosen, because it is whatever the key hashes to. The same property closes a first-contact attack: a HELLO whose sender identifier is not the hash of the key it carries is rejected, so a peer cannot claim someone else's address and have its own key trusted against it.

Every frame exchanged between clients carries an RSA signature, with one exception explained in Section 2.6. The signature is computed over a canonical encoding of the message that is deliberately not the wire encoding. It includes the message type by name and not by ordinal, so that appending a new message type later cannot change what an existing signature covers. It also includes the receiver identifier, which stops a relay from taking a frame that one peer genuinely signed for a third party and presenting it to someone else; the recipient checks that the receiver identifier is its own.

Trust in a peer's key is established on first use. The first key seen for an identifier is pinned. If a different key later appears for that identifier, the client does not accept it silently: it reports a key change and refuses to send until the user has re-verified. To verify, two users compare safety numbers, which are renderings of each side's key fingerprint in five-character groups, over a channel they already trust. Display names, by contrast, prove nothing. They are self-asserted, and are stripped of control characters and bidirectional overrides before display so that a name cannot forge the appearance of surrounding interface text.

2.6 Wire Protocol

Everything that crosses a socket is wrapped in a frame, shown in Fig. 2.3. A frame is a four-byte length, a one-byte frame version fixed at 1, and a payload that is an encoded message. The length must be between zero and one mebibyte, and a declared length outside that range is rejected before anything is allocated. A reader that trusts a declared length is one read away from an allocation chosen by the sender, so the check precedes the allocation. Frames are written whole and flushed, and writes are synchronised so that two threads cannot interleave halves of two frames.

Frame (the unit written to the socket)

length int32	version int8 = 1	payload (an encoded message) length bytes, 0 to 1 MiB
-----------------	---------------------	--

a declared length outside 0 to 1 MiB is rejected before anything is allocated

Message (the payload of a frame): flat, length-prefixed, big-endian fields

type int32	proto version int32	message Id	sender Id	receiver Id	time stamp int64	iv 12 B	payload <= 64 KiB	signature <= 512 B
---------------	---------------------------	---------------	--------------	----------------	------------------------	------------	----------------------	-----------------------

ordinal 2 <=128 B <=128 B <=128 B ms or -1 or -1 or -1

every variable field has a cap checked before allocation; -1 means absent, and is not the same as empty

no Java serialization anywhere on this path

Fig. 2.3: Layout of a frame and of the message it carries

The message is a flat sequence of length-prefixed fields in big-endian order, listed in Table 2.5. Every variable-length field has a cap that is checked before the array is allocated, which is what prevents a twenty-byte frame from becoming a request for gigabytes. A length of minus one means that the field is absent, and this is distinct from an empty field. There is no Java serialization anywhere on this path: deserializing untrusted bytes with the standard object reader is a recognised class of remote-code-execution defect, and the safest way to avoid it is not to use the mechanism.

Table 2.5: Fields of an encoded message

No.	Field	Encoding	Limit
1	type	int32, the enumeration ordinal	Must name a known type
2	protocolVersion	int32	Currently 2
3	messageId	int32 length and UTF-8, or -1	128 bytes
4	senderId	int32 length and UTF-8, or -1	128 bytes
5	receiverId	int32 length and UTF-8, or -1	128 bytes
6	timestamp	int64, milliseconds since the epoch	Within five minutes of local time
7	iv	int32 length and bytes, or -1	Exactly 12 bytes when present
8	payload	int32 length and bytes, or -1	64 KiB
9	signature	int32 length and bytes, or -1	512 bytes

Because the codec writes the ordinal of the message type, the order of the type constants is part of the wire format. New types are appended and nothing is reordered or removed. The

thirteen types are listed in Table 2.6.

Table 2.6: Message types

Ordinal	Type	Carries	Signed	Encrypted
0	HELLO	Identity key and display name	Registration only	No
1	HELLO_ACK	Nothing; the relay confirms a registration	No	No
2	KEY_EXCHANGE_INIT	Diffie-Hellman public key	Yes	No
3	KEY_EXCHANGE_REPLY	Diffie-Hellman public key	Yes	No
4	KEY_EXCHANGE_REJECT	Nothing	Yes	No
5	TEXT_MESSAGE	The message body	Yes	Yes
6	DELIVERY_ACK	The acknowledged message id	Yes	No
7	DISCONNECT	Nothing	No	No
8	ERROR	A short reason, such as RECIPIENT_OFFLINE	No	No
9, 10	PING, PONG	Nothing	No	No
11	TYPING	One byte: composing or stopped	Yes	No
12	READ_RECEIPT	Nothing	Yes	No

Every message between clients must carry a signature, and a frame that does not is dropped. This includes the acknowledgement and status types, which look inert but are not. An unsigned delivery acknowledgement or read receipt would let a relay show a message as delivered or read when it was neither, and an unsigned rejection would let it tear down a handshake without holding anyone's key. Typing notifications and read receipts are signed but not encrypted: they carry no content, and the relay already learns who is talking to whom from the routing fields, so encrypting them would hide nothing. HELLO is the one exception on the client-to-client path, because it carries the identity key and so there is nothing yet to verify it against; it is bound to its sender by the identifier-to-key rule instead.

2.7 Connection Establishment and Handshake

The sequence from first connection to first encrypted message is shown in Fig. 2.4. A client first registers with the relay by sending a HELLO that has no receiver. This registration is signed and carries the identity key from which the claimed identifier was derived, so the relay can check that whoever is asking for an identifier actually holds it. Without this check, a hostile party could connect as a victim and cause the real owner to be refused on the next login. The relay answers with a HELLO_ACK.

A client is deliberately not considered connected until that acknowledgement arrives. A socket that is merely open says nothing about whether the relay will route to the identifier yet, and a client that reported itself connected too early could send a handshake into that window and have it refused. A registration that is not acknowledged within ten seconds is treated as a failed attempt, closed, and retried under exponential backoff.

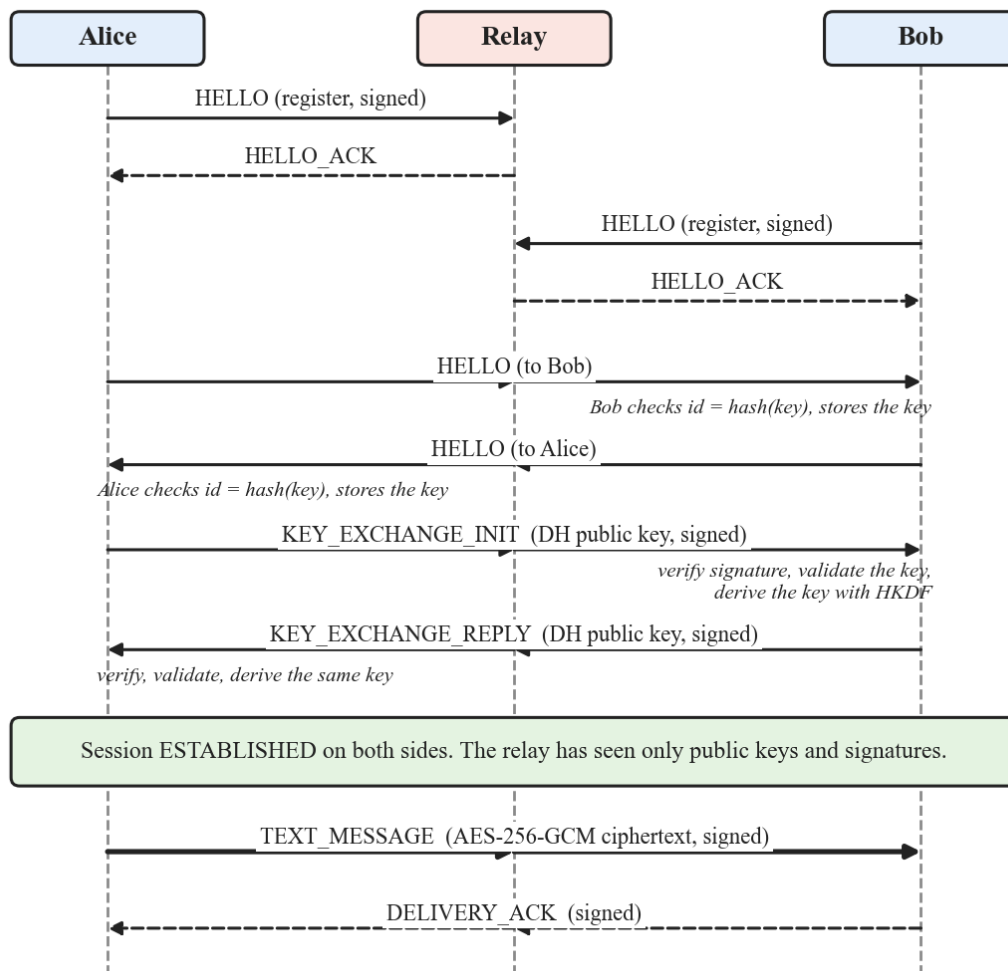


Fig. 2.4: Registration, mutual introduction, signed key exchange and first message

After registration the two peers introduce themselves with HELLO frames addressed to each other, and each checks that the sender identifier equals the hash of the key it carries before storing the key. The initiator then sends a KEY_EXCHANGE_INIT carrying its Diffie-Hellman public key, signed with its identity key. The responder verifies the signature against the stored identity key, validates the public value, derives the session key, and replies with a signed KEY_EXCHANGE_REPLY. The initiator performs the same checks and derives the same key. From this point the session is established. The relay has seen only public keys and signatures, none of which allow it to compute the session key or to substitute its own.

2.8 Message Processing and Session Management

2.8.1 The sending and receiving pipelines

Fig. 2.5 shows what happens to a message on its way out and on its way in. On the sending side the text, together with any reply fields, is encoded into a body, encrypted with AES-256-GCM using the next counter value as the nonce, wrapped in a message with its identifiers, timestamp and initialisation vector, signed, framed and written to the TLS connection. The reply metadata, meaning which message is being quoted and a preview of it, is part of the encrypted body, so the relay cannot see who is quoting whom.

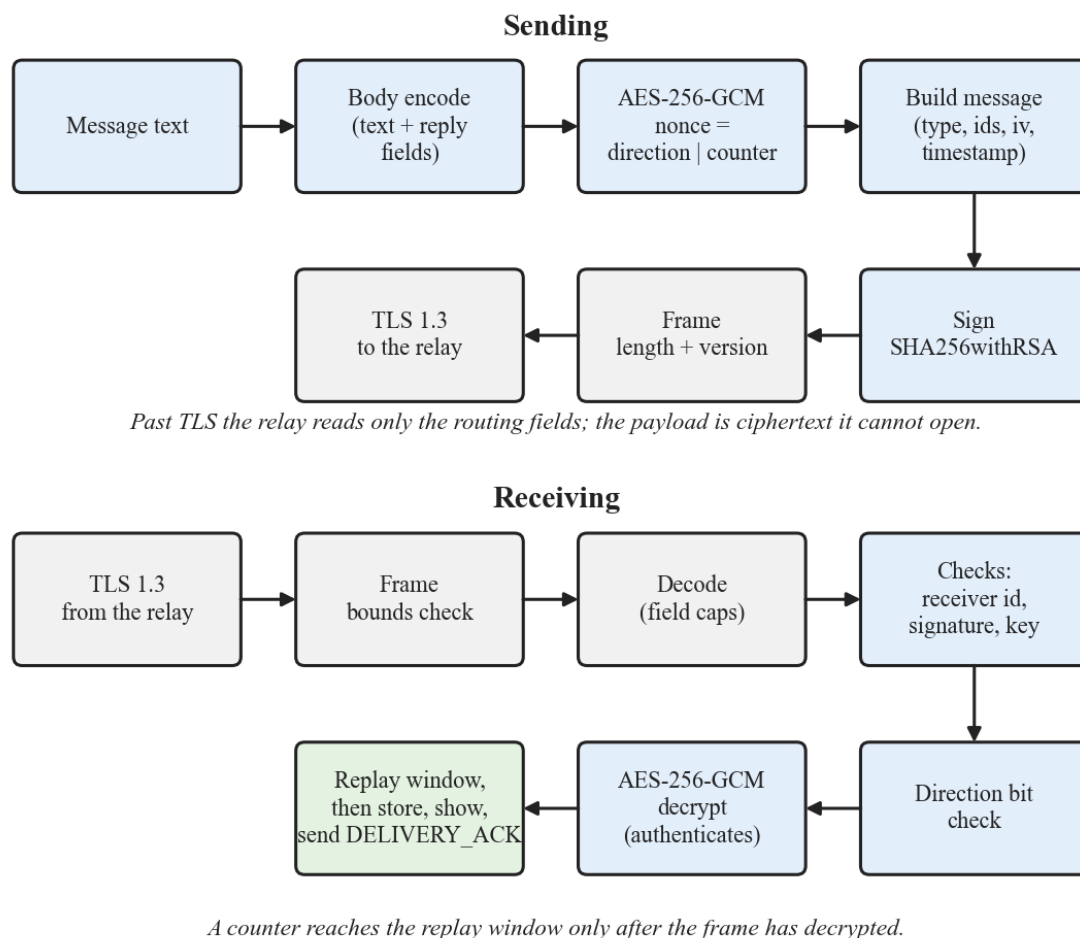


Fig. 2.5: Pipeline for an outgoing message (top) and an incoming message (bottom)

The receiving side applies the checks in a deliberate order. The frame length is checked before anything is allocated; the fields are decoded under their caps; the receiver identifier must be the client's own; the signature is verified against the sender's stored key; and the direction value must be the one the sender should be using. Only then is the ciphertext decrypted, and GCM authenticates as it decrypts. Registration of the counter in the replay

window comes last. This ordering matters. A counter that has been consumed cannot be used again by genuine traffic, so registering it before the frame had been authenticated would let a frame of pure noise spend a counter and cause the real message carrying that counter to be dropped. Verifying the signature before any of this prevents a forged frame from consuming a counter for the same reason.

2.8.2 Replay protection and key lifetime

Each session records the highest counter it has accepted and remembers the individual counters it has seen above a floor that sits 1024 below that highest value (Fig. 2.6). A counter at or below the floor is too old to be told apart from a replay and is dropped. A counter above the floor that has been seen before is a replay and is dropped. Anything else is accepted, so ordinary reordering inside the window still delivers.

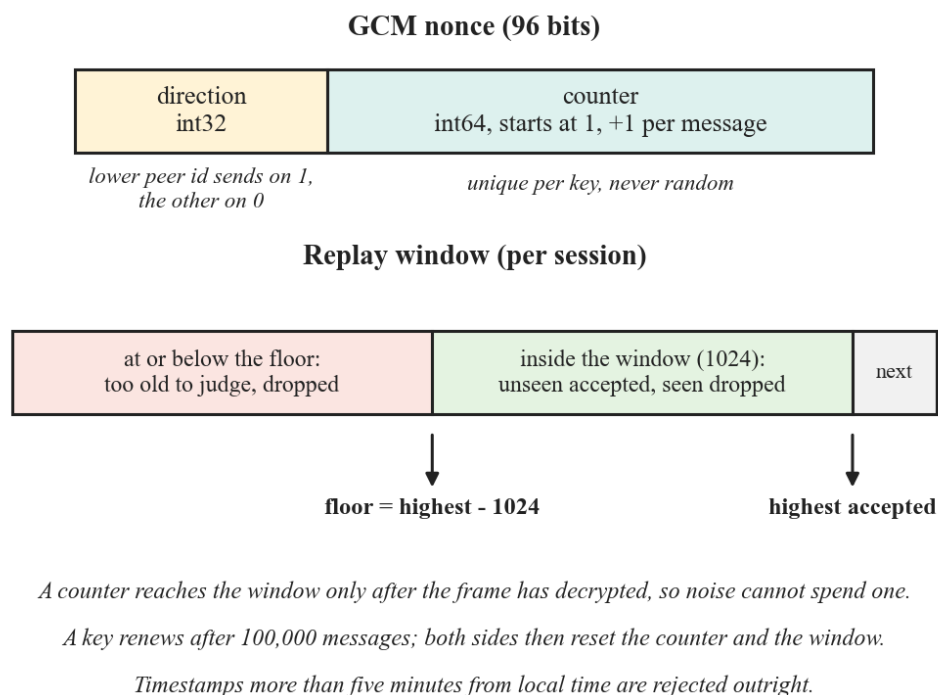
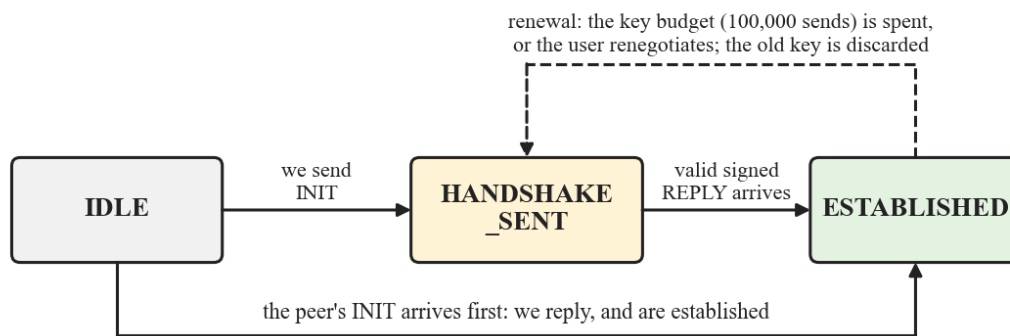


Fig. 2.6: Structure of the GCM nonce and the replay window

The floor is what makes the scheme correct. An earlier version of the code kept a set of seen counters, capped at 1024 entries with the oldest evicted, and had no floor. Once 1024 further messages had passed, the earliest counter was forgotten and a captured frame carrying it was accepted a second time. Protection was bounded by how many messages had arrived rather than by how old the frame was, which is not what a sliding window means. This was found by reviewing the claim made in the documentation, corrected with a floor, and the tests that now cover it were written first and watched failing against the old implementation. The cost

of the fix is that a genuine frame more than 1024 messages behind is also dropped; that is acceptable because a message whose timestamp is more than five minutes from local time is rejected anyway.

A key is also bounded by volume. After 100,000 sends the client discards the key and runs a fresh handshake, and both sides reset their counters and their replay windows when they adopt the new key, since nonce uniqueness is a property of a key and not of a session. The budget is far below any limit AES-GCM imposes; its purpose is to bound how much a single compromised key would reveal. The renewal is not seamless: while it is in flight the session is not established, so sending is refused, and a message already on the wire under the old key will fail authentication at the far end. A counter is also constrained to the range from 1 to the budget, because a single frame carrying an enormous counter value would otherwise move the replay floor beyond everything a peer would ever send and silently end the conversation.



A session is not established while a renewal is in flight, so sending is refused until the new key lands.

EXPIRED exists in the code and is never set: there is no timer-based expiry.

Fig. 2.7: Session states and transitions

The session states are shown in Fig. 2.7. A fourth state, EXPIRED, exists in the code and is never set, because there is no timer-based expiry; the limitations section in Chapter 4 states this.

2.9 Relay Server Design

The relay is intentionally small. It accepts TLS 1.3 connections only, and a plaintext connection is dropped. The first frame on a connection must be a HELLO registration, and anything else before it closes the connection. The registration is verified before it is honoured: the claimed identifier must be the hash of the key in the frame, the signature must verify against that key, the timestamp must be within a minute, and the message identifier

must not have been used before. A failure is answered with an `ERROR` carrying a specific reason and the connection is closed. A duplicate identifier is refused with `ID_TAKEN`.

Once registered, a connection may only send frames whose sender identifier is the one it registered under. A frame that claims another sender is refused with `SENDER_MISMATCH` and the connection is closed. The relay routes by receiver identifier, and answers `RECIPIENT_OFFLINE` when nobody is registered under it. It answers `PING` with `PONG` and disconnects idle connections. It never inspects a payload, never stores a message, and queues nothing for an absent peer: a message addressed to someone who is not connected is simply not delivered by the relay, which is why the client keeps its own outbox (Section 2.10).

Abuse is limited with a total connection cap (500 by default), a per-address cap (10), a token-bucket rate limit per connection, and a handshake timeout, all configurable. Operational counters are exposed on a plain-text metrics endpoint that is bound to the loopback address only, because it is unauthenticated. The relay's TLS key is held to the same standard as the clients' trust: a packaged relay refuses to start unless a keystore is configured, and the development keystore is excluded from both the packaged jar and the container image. This rule exists because the development key was once found inside the relay's own jar and was also the default keystore path, so an operator who simply started the server would have served a key that anyone can regenerate.

2.10 Client Design: Storage, Keys and Delivery State

Fig. 2.8 shows how the passphrase leads to the two secrets that matter and what is stored in the profile directory. The passphrase is typed by the user and is never stored. It unlocks the PKCS#12 keystore that holds the identity key pair, and, separately, it is stretched with PBKDF2-HMAC-SHA256 at 210,000 iterations over a random per-profile salt to produce the key that encrypts the message database. The salt and the iteration count are kept in the profile so that the count can be raised for new profiles without making existing databases unreadable.

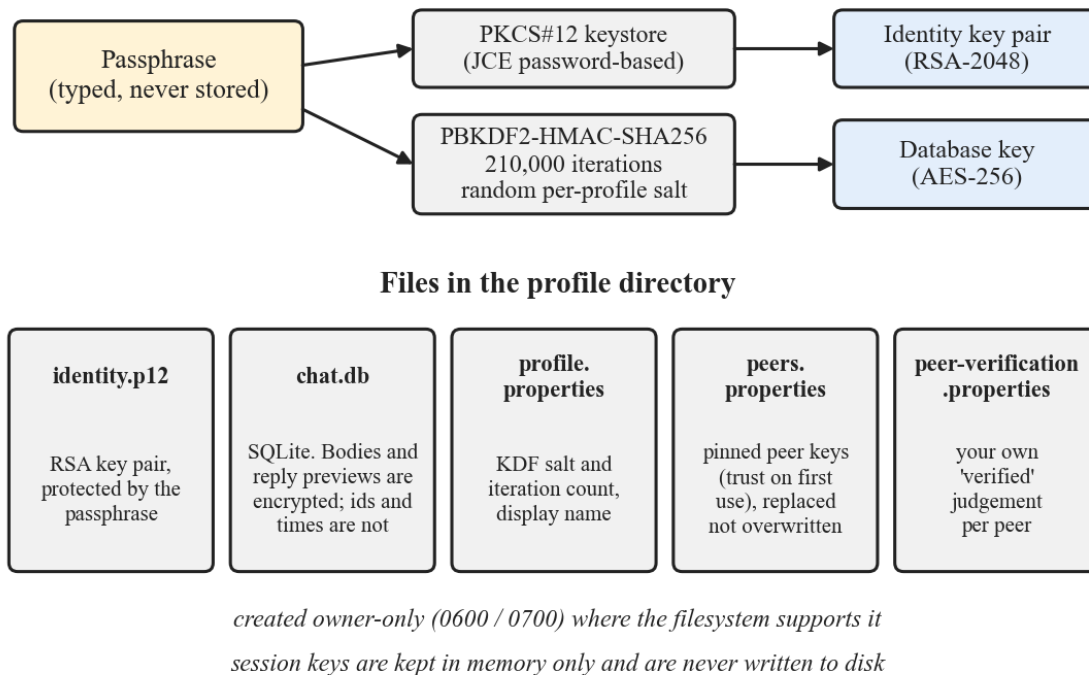


Fig. 2.8: Key derivation and the contents of the profile directory

Identity keys are generated inside the client process. An earlier version generated them by running the JDK keytool utility in a subprocess and passing the passphrase on its command line, which published the single most valuable secret the client holds to the process list of the machine for as long as generation took. It also made a packaged client unusable on a machine without a full JDK. Generation in process removes both problems. The identity keystore, the pinned peer keys and the directory that holds them are created readable only by their owner wherever the filesystem supports it. Session keys are held in memory only, are never written to disk, and are lost when the program exits. The pinned peer-key file is written by replacing it, so an interrupted write cannot leave a half-written store, and on startup every stored entry is decoded back into a key before the client will run. A store that is present but damaged stops the client instead of silently emptying it, because starting on a damaged store would reset trust: known contacts would become strangers and the key-change warning could not fire.

Only the message bodies and the quoted-reply previews are encrypted in the database. Participants, timestamps and message counts are visible to someone who holds the file. This partial encryption is a stated limitation, and the strength of what is protected rests entirely on the passphrase: no iteration count rescues a guessable one.

Delivery state is a small ladder: a message is first pending, then sent, then delivered, then

read. A message is pending when it has been written down on this device but not yet handed to the relay, because the transport was down or no session with the peer existed yet; it is shown with a clock rather than a tick, since a tick says sent and the message has not left. A pending message is retried by the client when a session is established and when the connection is restored. A message becomes sent when the transport accepts it, delivered when the peer's client acknowledges it, and read when the peer opens the conversation. A fifth state, failed, sits off the ladder: a message that failed for a reason retrying will not fix is marked so, and can still become sent if it later goes out. The ladder only ever moves forward. Because the client acknowledges a message every time it is received and the relay may redeliver, an acknowledgement can arrive after a read receipt has already been applied; without a guard that acknowledgement would pull a read message back to delivered, which the sender would read as the peer un-reading the message. The guard is applied both in storage and in what is drawn.

2.11 The Electron Client

2.11.1 Why Electron, and why the cryptography stays in Java

The first graphical client was written in Java Swing. It is complete, but building a modern, fluid interface in Swing means painting most components by hand, and animation and layout work that is routine in a web stack is laborious there. Electron provides an embedded browser engine and so a mature ecosystem for building interfaces. Moving to it, however, raises two risks, and the design is shaped by them.

The first risk is duplication. Rewriting the protocol and cryptography in TypeScript would create a second implementation of exactly the code this project is most concerned to keep single, and would put AES-GCM, the signed handshake and the replay window into new, untested code for no benefit the user could see. It would also require re-implementing the reading of every existing file format byte for byte. The design therefore keeps the Java client logic and the shared core unchanged, running as a headless child process called the engine, and uses Electron only to draw the interface. An existing user's identity keystore, message database and peer files continue to work with no migration.

The second risk is new, and Swing never had it. The interface displays text written by other people, and it does so in a browser engine. In an Electron application, a cross-site-scripting defect in a chat transcript is not merely a display problem; it can become execution of code on the user's machine. The design therefore treats the renderer as hostile territory: keys,

decryption and the passphrase-derived secrets stay in the Java process, and the renderer receives already-decrypted text only for display.

2.11.2 Process architecture

The application has three layers (Fig. 2.9). The renderer is a sandboxed React application with no Node.js access and no key material. A preload script exposes exactly two functions to it, one to invoke a command and one to subscribe to an event. The main process, which runs in Node.js, starts the engine, supervises it, matches each reply to the command that caused it, and forwards events to the renderer. The engine is the Java process: it reuses the existing client classes and the shared core, and talks to the relay over TLS exactly as the Swing client does.

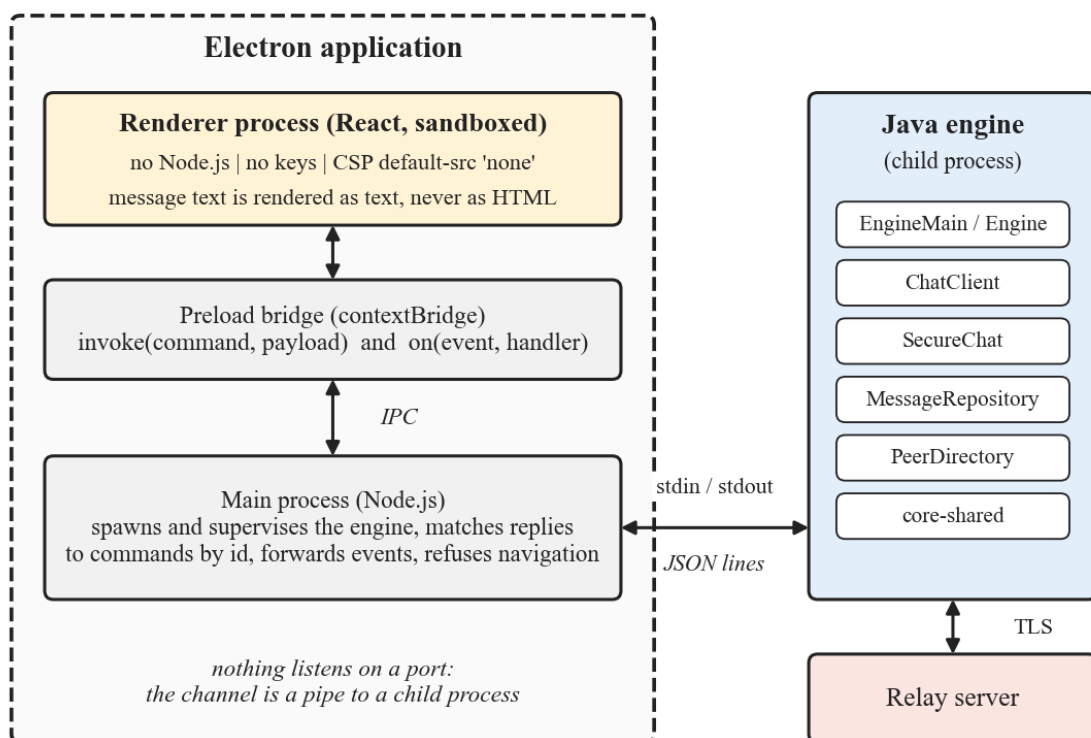


Fig. 2.9: Process architecture of the Electron client

The channel between the main process and the engine is a pipe on the engine's standard input and output, carrying one JSON object per line, and not a network socket on the loopback interface. The passphrase, every plaintext message and the identity all cross this channel. A loopback port is reachable by every other process running as the same user, whereas a pipe to a child process is private to its parent by construction. The engine opens no listening socket of any kind.

Standard output belongs to the protocol. A single stray print statement anywhere in the process, in the engine, in the client code or in a library it uses, would put a malformed line into the stream and desynchronise the caller. The engine therefore binds its frame writer to the process's output file descriptor before anything else runs and then redirects the ordinary standard output to standard error, so that everything that believes it is printing to the console lands harmlessly in the log. Writes to the stream are synchronised, because events arrive on the transport thread while replies are written by the command loop. This property was verified by observation: every line of the engine's logging, including warnings from the JVM and from the SQLite library, appeared on standard error while the frame stream stayed clean.

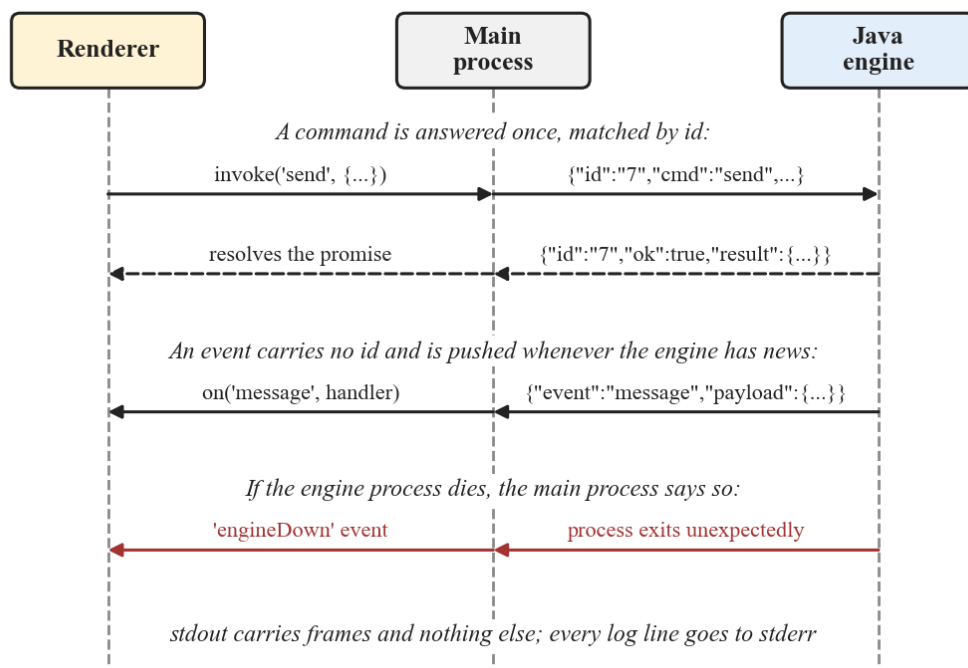


Fig. 2.10: Commands, replies and events between the renderer, the main process and the engine

2.11.3 The command and event interface

The interface between the renderer and the engine is defined once, in a typed TypeScript file that the main process, the preload script and the renderer all import, so that the three agree on the shape of every frame. A command carries an identifier and a name, and the engine answers with the same identifier. Events carry no identifier and are pushed whenever the engine has news. Fig. 2.10 shows the three kinds of message and the processes between which they pass, and the commands are listed in Table 2.7.

Table 2.7: Commands accepted by the engine

Command	Purpose
---------	---------

Command	Purpose
status	Report whether an identity exists, whether it is unlocked, the relay address and the connection state
unlock	Open an existing identity, or create the first one, from a passphrase and display name
connect, disconnect	Open or close the connection to the relay
listConversations	List conversations with preview, unread count and verification state
history	Return the messages of one conversation, newest first up to a limit
send	Send a message, optionally quoting an earlier one
startSecureChat, renegotiate	Begin or restart the key exchange with a peer
fingerprint, setVerified	Return the two safety numbers; record the user's verification decision
markRead, readReceipt, typing	Clear unread state; tell the peer a conversation was read or is being typed in
search	Search every stored conversation for a string
setActivePeer, shutdown	Note which conversation is open; stop the engine cleanly

The engine pushes events for an incoming message, a change in delivery state, a read receipt, a typing notification, a change in connection state, and an error from the relay. The main process adds one more, engineDown, if the engine process exits unexpectedly, so that the user interface can show that nothing can be sent and not appear to be working while silently stalled. Policy is deliberately kept out of the engine: it reports that a message arrived, and it is for the interface to decide whether to notify the user, because only the interface knows what is on screen.

Three implementation points came from problems met in practice. The engine is started by invoking the Java runtime directly with a wildcard class path and not through the build tool's generated batch script, because current versions of Node.js refuse to spawn a batch file without a shell and enabling a shell would add quoting rules and a command-injection surface for no benefit. The engine's working directory is set explicitly, because in development it locates the pinned development certificate relative to where it is running; inheriting the caller's directory made trust depend on where the application happened to be launched from, and this was found when an end-to-end test failed with neither client able to connect. And command names are validated before the check that an identity is unlocked, because a misspelled command otherwise reported that the engine was locked and sent the caller to debug the wrong problem.

2.11.4 Hardening of the renderer

The measures that confine the renderer are summarised in Table 2.8. Each is load-bearing: relaxing any one of them widens what an injected script could reach. None of them replaces

the discipline of never rendering message text as markup; they are the second line of defence behind it.

Table 2.8: Measures that confine the renderer

Measure	Effect
Context isolation on, Node.js integration off, sandbox on	The page cannot reach Node.js or the file system; it sees only the preload bridge
Preload bridge of two functions	The whole surface a script in the page can use is one command call and one event subscription
Content-Security-Policy default-src 'none', connect-src 'none'	The page can load only its own scripts and styles and can make no network request; the network is the engine's job
No remote content, fonts or analytics	Nothing is fetched from any host; the page works offline and discloses nothing by loading
Navigation refused; new windows opened only for https links in the system browser	A link in a message can never replace the application page or open inside it
Message text rendered as text nodes; search highlights built from text segments	Markup in a message is displayed literally and never interpreted
Keys, decryption and passphrase-derived secrets stay in the engine	The renderer holds no key material and no plaintext at rest

2.12 User Interface Design

The interface is a React application written in TypeScript and bundled with Vite. All its state is held by a single hook that subscribes to the engine's events and issues commands. The engine is treated as the source of truth: after anything that changes stored state, the conversation list is read again from the engine and not patched locally, so unread counts and previews cannot drift from the database. Only the open transcript is updated optimistically, because that is where waiting for a round trip would be visible.

2.12.1 Layout, appearance and motion

The window is divided into a 320-pixel sidebar and a chat pane. The sidebar holds the search box, the new-chat control and the conversation list; the settings panel slides over it so that the conversation being read remains in view. The chat pane has a header, the transcript, a composer and an optional safety-number panel. Each person is given a circular avatar with their initials and one of seven gradients chosen by a hash of their peer identifier, so the same person keeps the same colour on every launch. Initials and colour are computed by a single function that both the list and the header use; an early version computed them separately and drew the same person differently in the two places.

The appearance is a cool graphite base with one vivid accent, used only where it carries meaning: the primary action, read ticks, the verified shield and keyboard focus. Outgoing

messages are a lighter neutral and not an accent colour, so that a transcript is two quiet greys with a bright colour appearing only where it signals something. Table 2.9 gives the principal colours for the dark and light themes, both defined as style variables and switched at run time. The stylesheet names a typeface first and falls back to the system fonts; the first font is not bundled, so on the test machine the interface is rendered in Segoe UI.

Table 2.9: Principal colours of the two themes

Role	Dark	Light
Page background	#0B0D10	#F7F8FA
Surface (sidebar, header)	#111419	#FFFFFF
Incoming message	#171B22	#F1F3F6
Outgoing message	#232A35	#E4E9F0
Primary text	#E8EBEF	#0F1319
Secondary text	#8B94A3	#5E6875
Accent	#22D3EE	#0891B2
Unread badge	#FF6B57	#F0553D
Error	#FF6B6E	#D93A3D

Motion is restrained and functional. Only a message that has just arrived animates, with a short rise and fade of about 180 milliseconds, and the view glides to the foot of the transcript. Opening a conversation animates nothing, because fading in a whole history at once reads as the screen flickering. A setting turns all animation off, and the operating system's reduced-motion preference is honoured as well. The unread badge is deliberately not the accent colour, so that a count of unread messages does not compete with the accent for attention.

2.12.2 Behaviour

- **Sign-in.** The first run asks for a display name and a passphrase and says, before the passphrase is chosen, that it cannot be recovered. Later runs ask only for the passphrase. A wrong passphrase is refused without saying which check failed.
- **New conversation.** A conversation is started by pasting a peer identifier. The identifier is validated in place, as 32 hexadecimal characters that are not the user's own, so that a mistyped value can be corrected without dismissing anything first.
- **Transcript.** Messages are grouped by day. Each sent message shows a state: a clock while pending, one tick when sent, two when delivered, two highlighted when read, and an exclamation mark if it failed. A typing indicator shows when the peer is composing.
- **Verification.** A panel shows the two safety numbers in groups of five characters so

that they can be read aloud and compared, with a control to record that they matched. A verified contact is marked with a shield in the list and the header. The decision is stored on this device only, apart from peer-asserted names, so a compromised names file cannot forge a shield.

- **Search.** A box at the top of the sidebar, reached with Ctrl+F, searches every conversation. It is debounced by 200 milliseconds, discards the response to any query that has been superseded, highlights the match by splitting the text and never by building markup, and reports three distinct states: still searching, nothing found, and the number of matches. Selecting a result opens that conversation and scrolls to the message, loading up to 5,000 messages if it is older than the usual window, and says so if it still cannot be reached.
- **Replies.** A button on each message, reachable by keyboard, starts a reply. A bar above the composer names the person being answered and Escape cancels it. The sent message carries a quote block attributed to its author, which scrolls to and highlights the original when clicked. A reply to a message the engine cannot find is refused and not sent without its quote.
- **Notifications.** When a message arrives while the window is not in front, one operating-system notification is raised for that person, replacing any earlier one. It names the sender. It shows the message text only if the user has switched previews on, and the default is off, because a notification can appear on a lock screen or be mirrored to another device by the operating system.
- **Settings.** Theme, reduced motion, the user's own peer identifier with a copy control and safety number, the relay address with an indication of how to change it, and the version. There is no update switch, because this client performs no update check, and a control wired to nothing would be a dead end.
- **Errors.** A relay error such as RECIPIENT_OFFLINE is translated into a sentence, and the interface does not promise what will happen to the message, because the relay's error carries no message identifier and so cannot be tied to one bubble.

2.13 Implementation Summary

Table 2.10 gives the size of each module in non-blank lines of source, excluding tests. The shared core and the relay together are about four thousand lines, which is small enough for one person to read in a day; the Swing client is larger because it paints its own components; the headless engine is under seven hundred lines because it reuses the client logic. The

Electron application, including its stylesheet and its helper tests, is about 2,600 lines.

Table 2.10: Size of each module (non-blank lines, excluding tests)

Module	Language	Source files	Lines
core-shared	Java 8	32	3,032
chat-server (relay)	Java 8	9	1,056
chat-desktop (Swing client and client logic)	Java 8	48	9,876
desktop-engine	Java 8	2	662
chat-desktop-electron	TypeScript, React, CSS	16	2,604
chat-mobile (early Android client)	Java	17	1,582

CHAPTER 3

RESULTS AND DISCUSSIONS

This chapter reports what was verified and what was measured. Each result is stated with the conditions under which it was obtained, and Section 3.6 says what the results do not show.

3.1 Test Strategy and Outcomes

Testing is organised in four layers. The first is unit tests of the shared core and of each module, written with JUnit. They cover the encoding and decoding of every message type, the cryptographic primitives against seventeen frozen test vectors, the replay window, the session send budget and key renewal. The second is integration tests in which a real relay and real clients exchange messages over TLS. The third is a scripted end-to-end run in which the Electron main process starts two engines and passes a message through a running relay. The fourth drives the real Electron window through the browser debugging protocol and checks what is drawn. Table 3.1 and Fig. 3.1 give the counts for the automated suites.

Table 3.1: Automated test results

Suite	Tests	Failures	Skipped
core-shared (unit)	151	0	4
chat-server (unit)	51	0	0
chat-desktop (unit)	170	0	0
chat-desktop (integration)	1	0	0
desktop-engine (unit)	8	0	0
JVM total	381	0	4
Renderer helpers (Node test runner)	18	0	0

The four skipped tests check file permissions that exist only on POSIX file systems, and were skipped because the tests were run on Windows. They have not been run for this report, so the behaviour they check (that the identity keystore and the profile directory are readable only by their owner) is unverified on this machine.

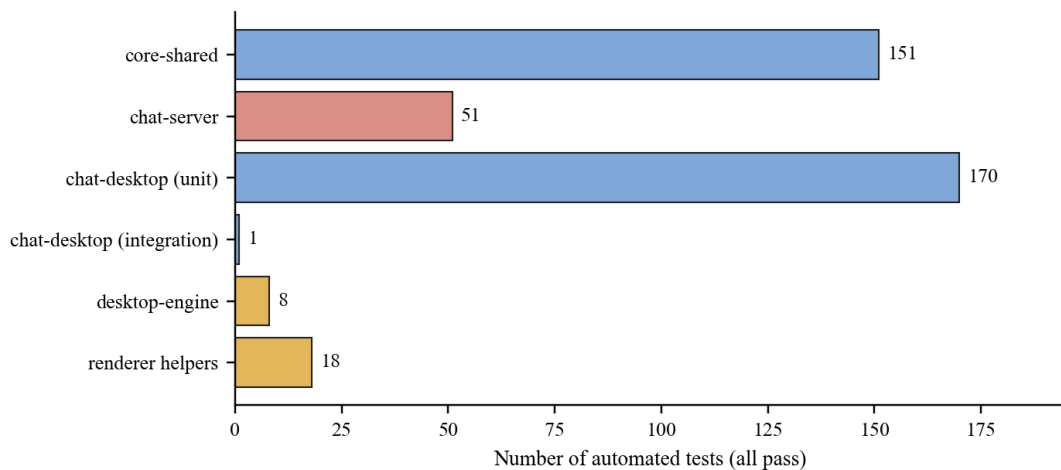


Fig. 3.1: *Number of automated tests per suite*

Two further checks were run against the running application. The scripted end-to-end run performs 17 checks, including that a message sent by one engine arrives at the other through the relay. The window-driven run performs 24 checks in the real Electron window, covering among other things sending, replying with a quote, jumping to the quoted original, and search with a single match. All 17 and all 24 passed in the final run. The window-driven run revealed a wrong expectation in the test itself, not in the application: it expected two search results for a word that appears in one message, and was corrected after confirming by hand that one was right.

Several guards were validated by mutation: the guard was removed and the corresponding test was confirmed to fail, so that a passing test means something. This was done for the release-build refusals in the TLS support and the relay configuration. It was not done systematically for every test, and no coverage figure is reported.

3.2 Security Evaluation

3.2.1 Defects found and corrected

A security design is better judged by the faults it records than by the claims it makes. Table 3.2 lists the principal defects found during development. Each was found by checking a claim, looking inside a built artifact or reading the code against its own documentation, and each now has a test or a verified change.

Table 3.2: Principal defects found and corrected during development

Defect	Consequence	Correction
Both peers built the GCM nonce with the same direction bit	Each side's n-th message reused a key and nonce pair, which breaks GCM	Direction derived from the peer identifiers; pinned by a test

Defect	Consequence	Correction
Replay window kept at most 1024 counters with no floor	A captured frame was accepted again after 1024 later messages	Floor at highest minus 1024; tests written first
At-rest key derived with HKDF	No work factor; passphrase guessed at hash speed	PBKDF2-HMAC-SHA256, 210,000 iterations; old databases re-encrypted
Peer identifiers were 28 bits and included the display name	Collisions near 16,000 users; renaming changed the address	128 bits derived from the key alone
Nothing bound an identifier to its key	A peer could claim another's address on first contact	Binding checked on every HELLO
Relay private key shipped inside its jar and was the default	An unconfigured relay served a key anyone can regenerate	Packaged relay refuses to start without a keystore
Packaged client trusted the development certificate	TLS in name only	Release builds refuse to connect without a truststore
Keys generated by running keytool with the passphrase on the command line	Passphrase visible in the process list; failed without a JDK	Keys generated in process
Send budget terminal and renewal did not reset the counter	Peer unreachable for the life of the process	Renewal resets counters on both sides

Not every corrected defect was found by a test. The nonce reuse, the relay key in its jar and the plaintext database on Android were found by examination, which is why Chapter 2 describes how each was found as well as what was done.

3.2.2 Static analysis

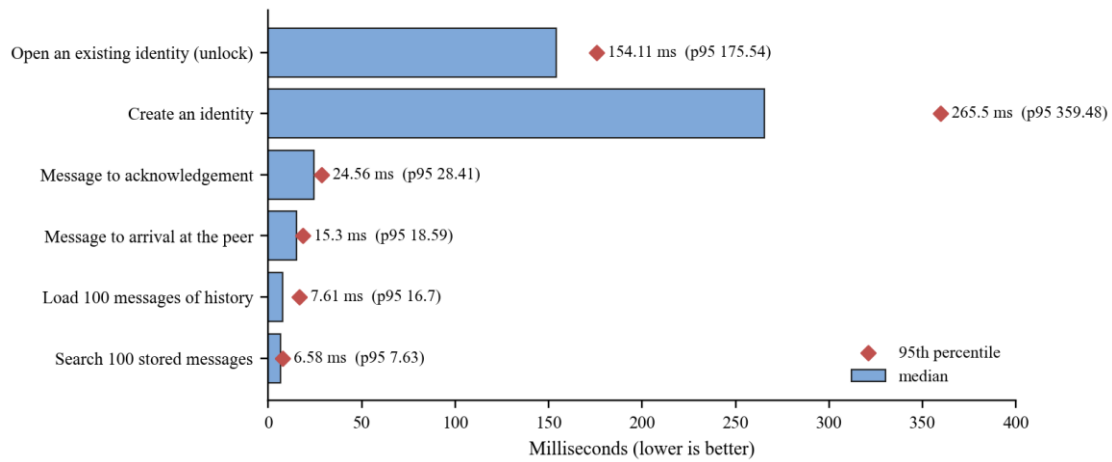
SpotBugs with the find-sec-bugs plugin runs on every build of every JVM module at high confidence, and a finding fails the build. There are no unresolved findings at that level and one scoped suppression. Removing two earlier pattern-wide suppressions exposed a real defect, in which the relay encoded one payload with the platform default character set; it was corrected and the suppressions removed. Lowering the threshold to report everything gives 216 findings in the main sources and 167 in tests. These were triaged by hand, mostly log injection on paths where the identifier was already validated, and none was judged exploitable. That judgement was made by the authors and is not an independent review.

3.3 Performance Measurements

The measurements in Table 3.3, which Fig. 3.2 plots, were taken with a script that drives two engines against a relay running on the same machine: an AMD Ryzen AI 9 HX 370 with 24 logical processors and 19.6 GiB of memory, running Windows 11 and Node.js 24. Because client and relay share a machine, the figures contain no network latency. They show the cost of the software, and they are not what a user on a real network would see.

Table 3.3: Measured latencies on the test machine (milliseconds)

Operation	n	Median	95th pct.	Max
Create an identity	5	265.5	359.5	359.5
Open an existing identity (unlock)	5	154.1	175.5	175.5
Register with the relay	2	307.2	307.2	307.2
Key exchange	1	19.8	19.8	19.8
Message to arrival at the peer	100	15.3	18.6	46.7
Message to delivery acknowledgement	100	24.6	28.4	54.9
Load 100 messages of history	20	7.6	16.7	16.7
Search 100 stored messages	20	6.6	7.6	7.6

**Fig. 3.2: Median and 95th percentile latency of the measured operations**

The unlock time is dominated by the passphrase stretching, and it is meant to be slow: 210,000 iterations cost roughly 150 ms here, which is the same cost an attacker pays per guess on comparable hardware. A message crosses the relay in a median of about 15 ms and is acknowledged in about 25 ms. The registration and key-exchange rows rest on two observations and one, so they are single observations and not distributions, and the 95th percentile shown for them is just the largest value seen. The search and history figures are for a database of 100 messages. No larger database, no concurrent clients and no relay under load were measured.

3.4 The Running Application

Figs. 3.3 to 3.8 are screenshots of the Electron client taken from the real window during the window-driven run, with two clients exchanging messages through the relay. The message text in them was written for the test.

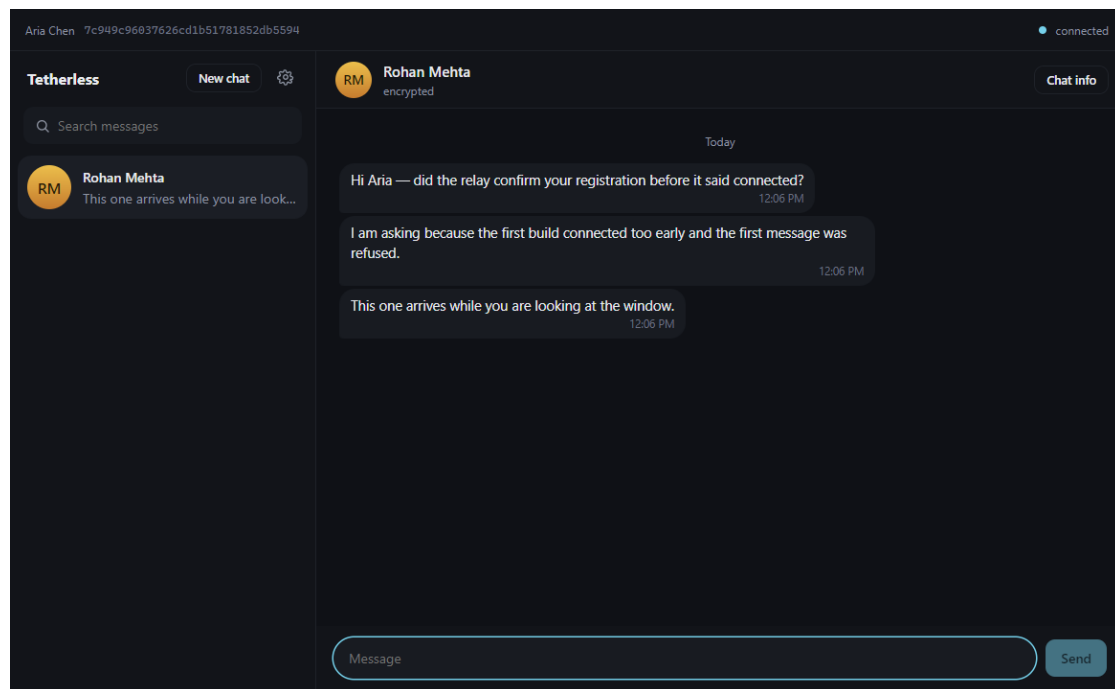


Fig. 3.3: *A conversation with message states and day grouping (dark theme)*

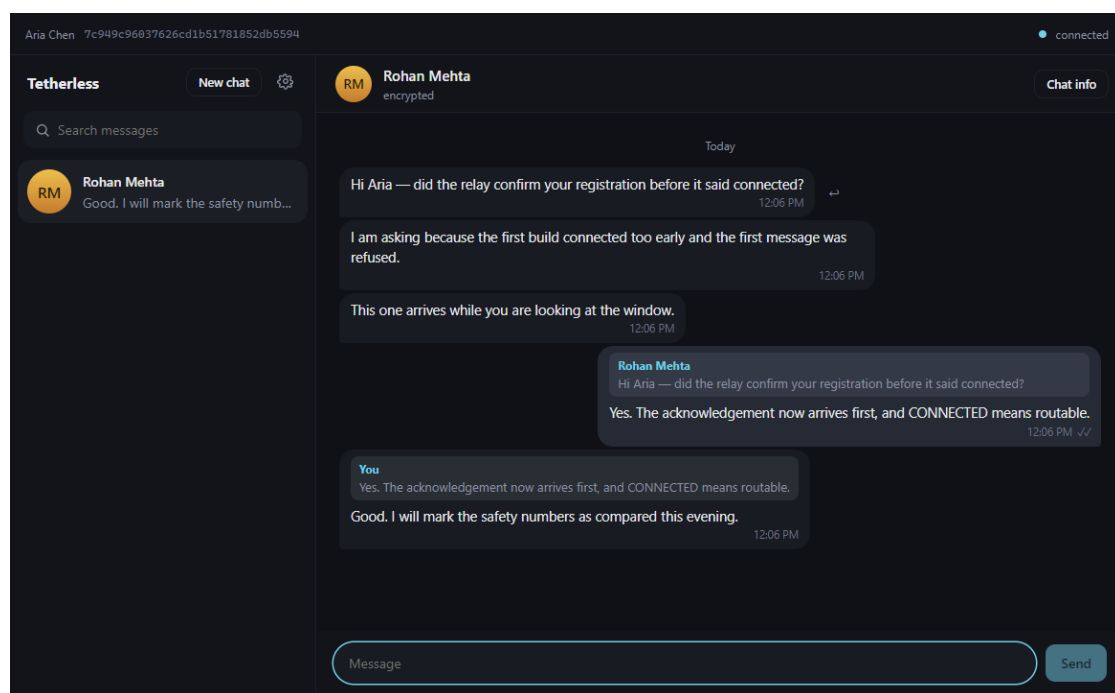


Fig. 3.4: *Replies showing the quoted original*

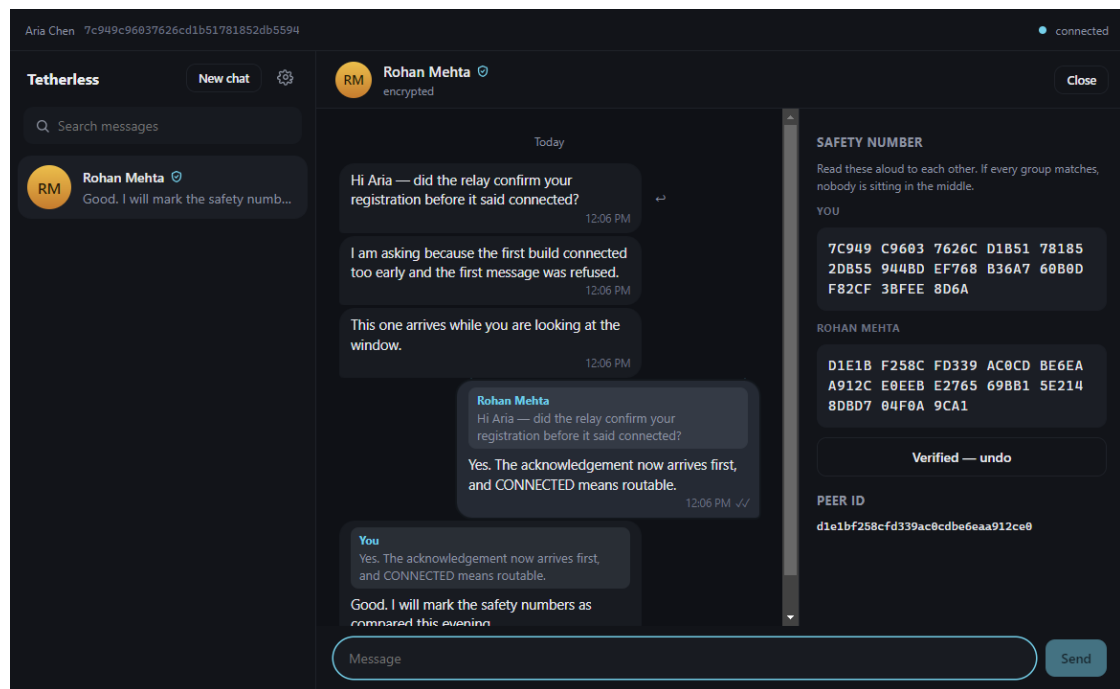


Fig. 3.5: Safety-number panel and the verified shield

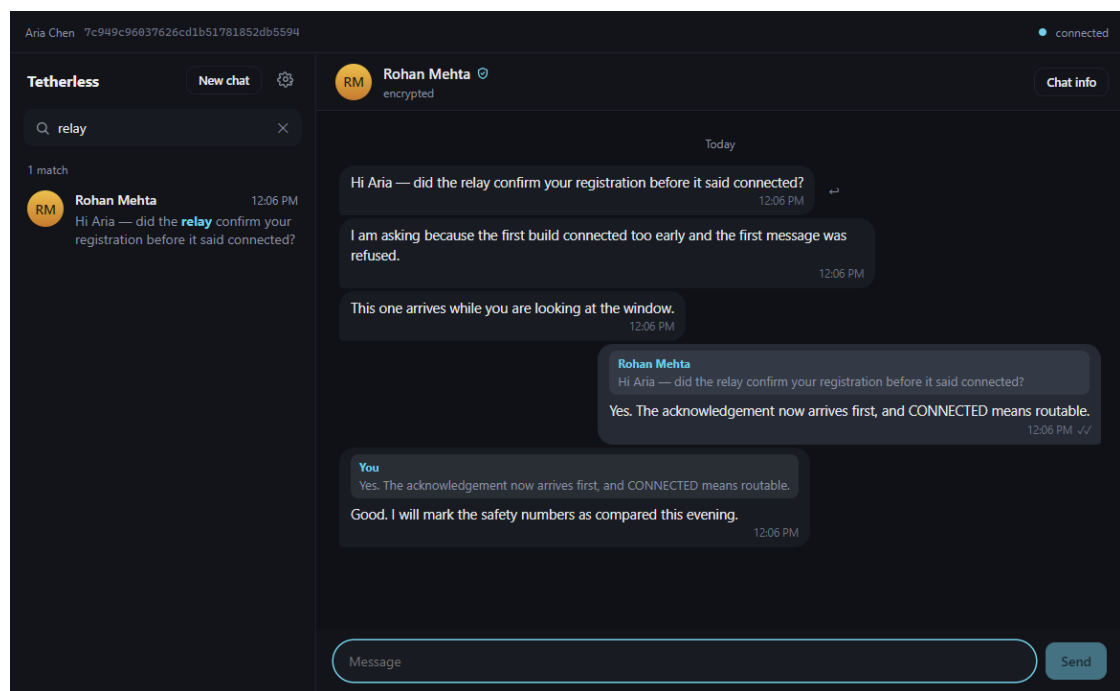


Fig. 3.6: Search results with highlighted matches

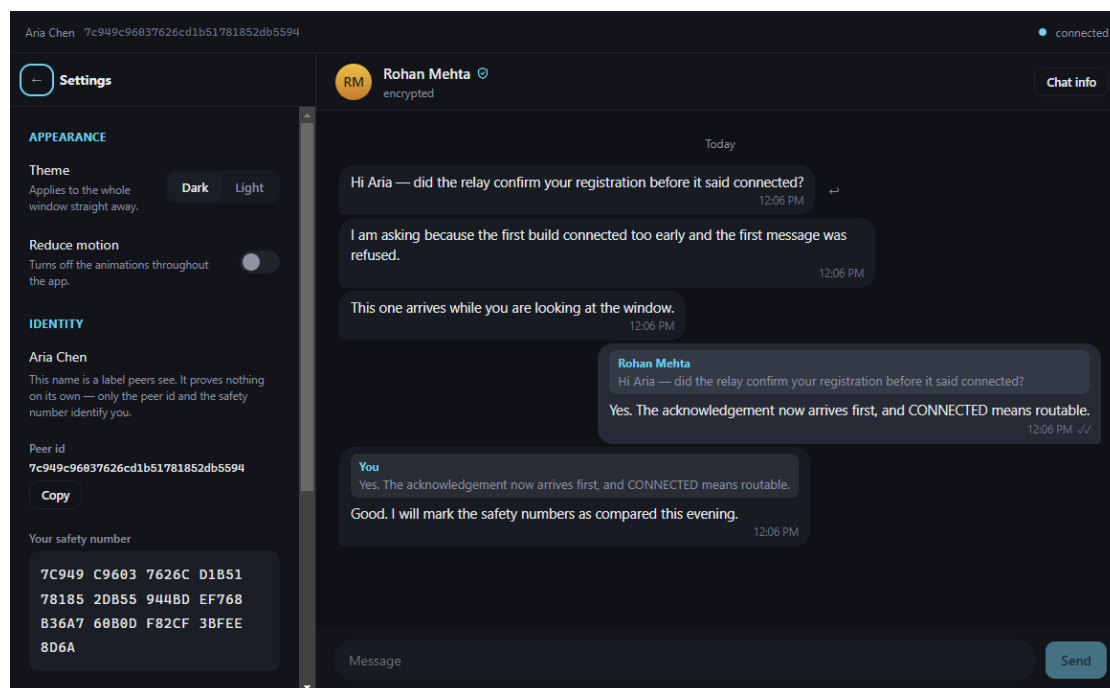


Fig. 3.7: *Settings panel (dark theme)*

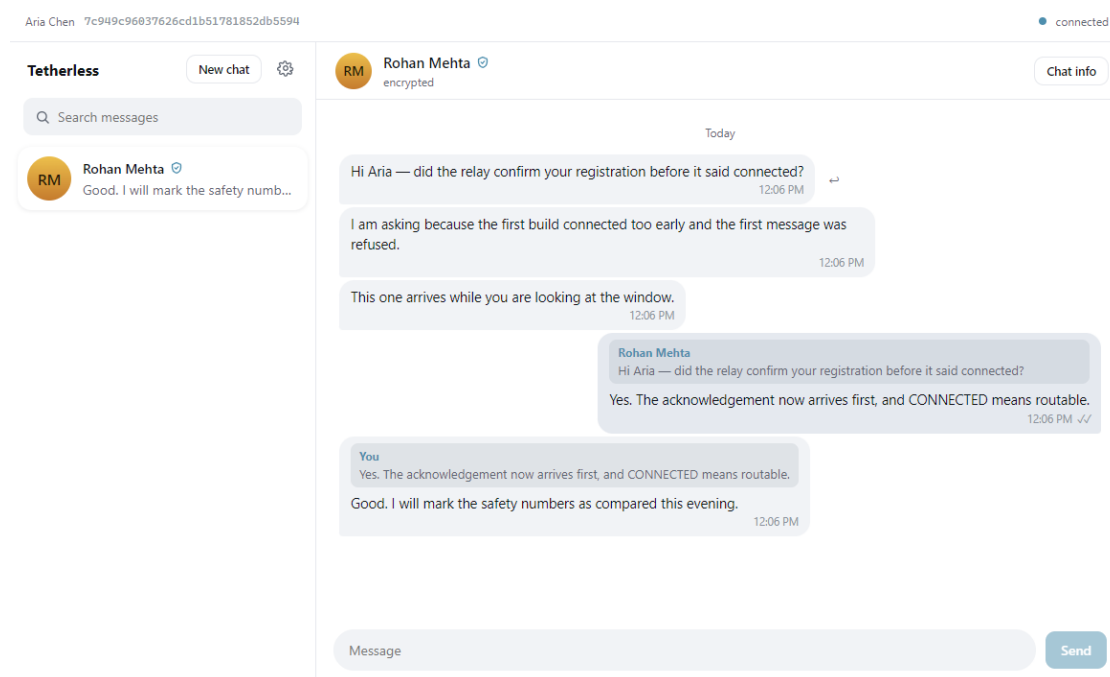


Fig. 3.8: *The same conversation in the light theme*

3.5 Discussion

Measured against the objectives in Section 1.3, the system delivers end-to-end encrypted one-to-one messaging in which the relay cannot read or forge a message, with identity verification through safety numbers, replay protection, encrypted history at rest and a desktop interface with search, replies and notifications. The relay was exercised as a hostile party in the integration tests, and no test found a way for it to read or alter a message.

The Electron client met its purpose: the interface is considerably easier to build and refine than the Swing one, and it did so without a second implementation of the cryptography, since the Java engine is unchanged logic behind a pipe. The cost is a larger installed size and a second process to supervise.

3.6 Limits of the Results

- All measurements are on loopback, on one machine, with small samples for the one-time operations.
- No usability study was carried out. The statements about the interface describe its design and not user experience.
- The tests were written by the same team that wrote the code, and no independent audit was done.
- Four POSIX permission tests were skipped on Windows and the behaviour they cover is unverified here.
- Only one relay and two clients were exercised together. Scale and behaviour under load are untested.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

This project set out to build a messenger in which the server that carries the messages is not trusted with their content. The result is a working system of a relay, a shared core library and desktop clients, the most recent of which is an Electron interface over a headless Java engine. Messages are protected with AES-256-GCM under a key agreed by a signed Diffie-Hellman exchange over TLS 1.3, contacts are verified through safety numbers, replays are rejected, and the history kept on the device is encrypted under a key derived from the user's passphrase. The system passes 381 JVM tests, 18 renderer tests, a 17-check end-to-end run and a 24-check run in the real application window.

The more instructive outcome was how the faults were found. Several of the most serious, such as the nonce reuse across directions, the relay key packaged in its own jar and the passphrase placed on a command line, were found by checking what the documentation claimed against what the code and the built artifacts did. The practice of writing down what the system does not protect, and of testing the guards that stop a development shortcut reaching a release, is as much a result of the project as the code.

4.2 Limitations

- **No forward secrecy beyond a handshake.** One key protects a whole session, so its compromise exposes every message in it. There is no Double Ratchet.
- **No metadata privacy.** The relay sees who talks to whom, when, and how large each message is.
- **Partial protection at rest on the desktop.** Only message bodies and reply previews are encrypted, and the protection is as strong as the passphrase.
- **No session expiry.** A key is renewed after 100,000 messages and not after any period of time.
- **One relay.** It cannot read messages but can refuse service and sees all metadata.
- **One-to-one text only.** No groups, no attachments and no multiple devices per identity.
- **Development certificate.** In development the relay's key can be regenerated by anyone; packaged builds refuse to use it.
- **No independent audit.** The tests and the review were done by the team that wrote the

code.

4.3 Future Scope

1. Replace the single session key with a ratchet so that each message has its own key, giving forward secrecy and recovery after a compromise.
2. Encrypt the whole message database, as the Android client does, so that participants and timestamps are protected as well.
3. Add session expiry by time, and make key renewal seamless so that no message is lost during it.
4. Support groups, using a protocol designed for them such as MLS [14], and support attachments.
5. Reduce what the relay learns, by padding messages and by moving towards several relays or direct connections between peers.
6. Add signed releases and an update mechanism that does not disclose the user's address to a third party.
7. Commission an independent security review, and run a usability study with real users.
8. Measure the system over a real network and under load, and with larger histories.

REFERENCES

- [1] W. Diffie and M. E. Hellman, "New directions in cryptography," IEEE Transactions on Information Theory, vol. 22, no. 6, pp. 644-654, 1976.
- [2] R. L. Rivest, A. Shamir and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," Communications of the ACM, vol. 21, no. 2, pp. 120-126, 1978.
- [3] T. Kivinen and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)," RFC 3526, IETF, May 2003.
- [4] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, May 2010.
- [5] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.
- [6] K. Moriarty, B. Kaliski and A. Rusch, "PKCS #5: Password-Based Cryptography Specification Version 2.1," RFC 8018, IETF, January 2017.
- [7] M. S. Turan, E. Barker, W. Burr and L. Chen, "Recommendation for password-based key derivation," NIST Special Publication 800-132, 2010.
- [8] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, August 2018.
- [9] N. Borisov, I. Goldberg and E. Brewer, "Off-the-record communication, or, why not to use PGP," in Proc. ACM Workshop on Privacy in the Electronic Society, 2004, pp. 77-84.
- [10] M. Marlinspike and T. Perrin, "The X3DH key agreement protocol," Signal, revision 1, November 2016.
- [11] T. Perrin and M. Marlinspike, "The Double Ratchet algorithm," Signal, revision 1, November 2016.
- [12] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt and D. Stebila, "A formal security analysis of the Signal messaging protocol," in Proc. IEEE European Symposium on Security and Privacy, 2017, pp. 451-466.
- [13] N. Unger, S. Dechand, J. Bonneau, S. Fahl, H. Perl, I. Goldberg and M. Smith, "SoK: Secure messaging," in Proc. IEEE Symposium on Security and Privacy, 2015, pp. 232-249.
- [14] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara and K. Cohn-Gordon, "The Messaging Layer Security (MLS) Protocol," RFC 9420, IETF, July 2023.